

4교시: Docker vs Local Computer

수업 목표

- Docker 실행과 local 실행의 차이를 compute, storage, network, configuration, observability 관점으로 비교한다.
- Docker를 쓰면 좋아지는 점과 나빠지는 점을 비용, 보안, 관리 효율 기준으로 판단한다.
- "언제 Docker를 쓰지 말아야 하는가"를 실습 난이도가 아니라 운영 적합성 관점에서 설명한다.

선행 지식

Week 1 개념	Docker 비교 지점
process	container process
OS/kernel	host kernel 공유, macOS의 Docker Desktop 내부 Linux VM 계층, Windows 사용자의 WSL 2 예외 경로
file path	image layer, volume, bind mount
localhost/port	host port, container port
env var/config	runtime environment injection
log/status	docker logs, status, exit code
cost/resource boundary	image size, disk usage, running container count

강의 전개

이 교시는 Docker를 홍보하는 시간이 아니라 도구 선택 기준을 만드는 시간이다. 학생이 Docker를 배우기 시작하면 모든 것을 container로 감싸고 싶어질 수 있다. 하지만 실제 운영에서는 Docker가 줄여주는 문제와 새로 만드는 책임을 함께 봐야 한다. 그래서 local 실행과 Docker 실행을 compute, storage, network, configuration, observability 관점으로 비교한다.

먼저 Week 1 local 실행을 복기한다. local process는 host OS에서 직접 실행되고, host file path를 읽고, host port를 사용하며, terminal에 log를 남긴다. Docker로 가면 같은 요소가 container process, image layer, volume, host/container port binding, docker logs로 이름을 바꾼다. 이름이 바뀌어도 underlying problem은 사라지지 않는다.

그 다음 Docker의 장점을 구체화한다. 장점은 "편함"이 아니라 재현성, dependency isolation, handoff, cleanup 가능성이다. Dockerfile과 run command가 명확하면 동료가 같은 실행 조건을 더 빨리 만들 수 있다. cloud resource를 만들기 전에 local container로 DB나 web server를 검증할 수 있으므로 비용 낭비도 줄일 수 있다.

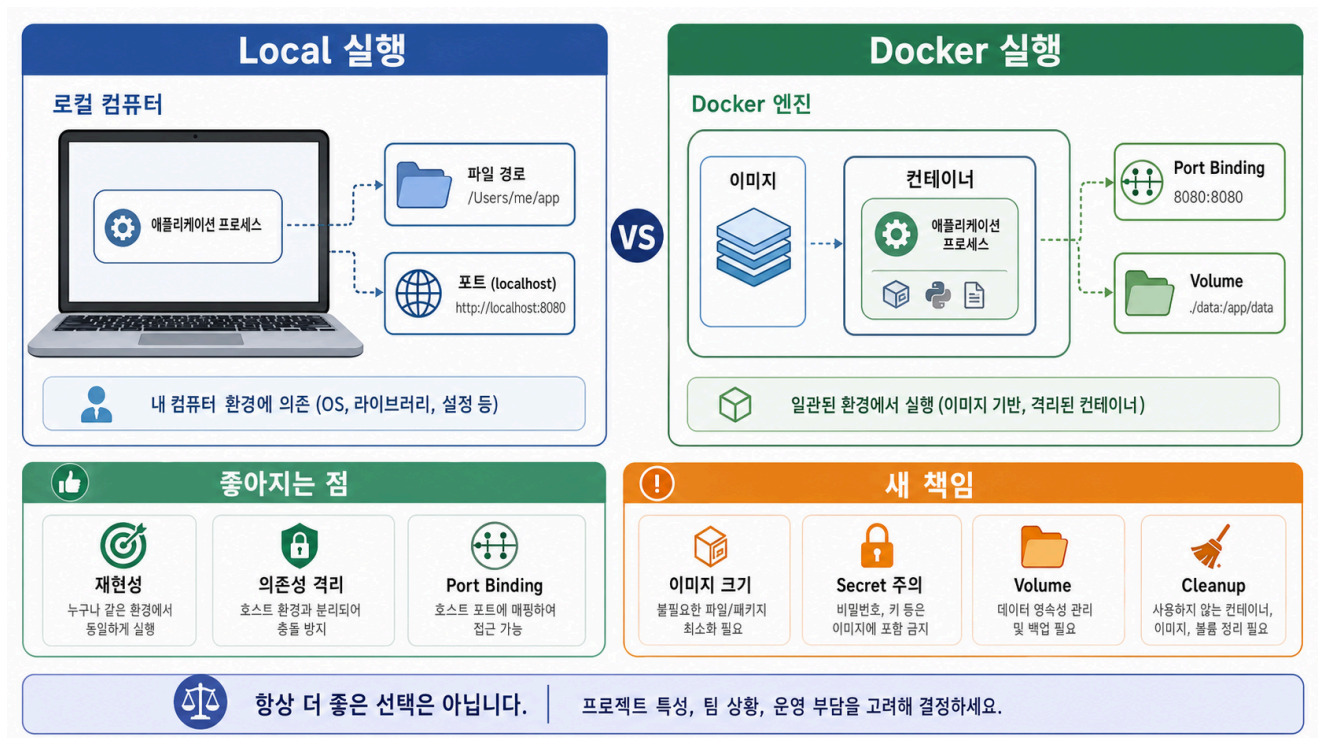
하지만 같은 비중으로 단점을 다룬다. image가 커지면 pull/build가 느려지고, container와 volume을 방치하면 disk와 port가 지저분해진다. secret을 build context나 image layer에 넣으면 지우기 어렵다. container 내부에서만 보이는 실패는 local보다 debugging이 어려울 수 있다. Docker 도입은 운영 책임을 없애는 것이 아니라 책임의 경계를 바꾸는 일이다.

수업 후반의 판단 활동에서는 "Docker를 써야 한다"가 정답이 아니다. 학생이 자기 Week 1 앱을 기준으로 실행 반복성, 의존성 복잡도, port 명확성, data persistence, secret 필요 여부를 평가하게 한다. Docker가 맞는 상황과 보류해야 하는 상황을 모두 말할 수 있어야 도구를 제대로 이해한 것이다.

local 실행 경험 복기

Week 1의 local 실행은 Docker를 배우기 위한 필수 준비였다. local에서 실행 조건을 설명하지 못하면 container 안에서도 같은 문제가 반복된다. Docker는 실행 환경을 표준화하지만, 어떤 파일을 넣을지, 어떤 command를 실행할지, 어떤 port를 열지, 어떤 설정을 외부에서 받을지는 사람이 결정해야 한다.

Visual 1: local 실행과 Docker 실행 비교 spine



이 이미지는 local computer의 process, file path, localhost port가 Docker 실행에서 image, container, port binding, volume으로 바뀌는 관계를 보여준다. 아래쪽의 "좋아지는 점"과 "새 책임"을 함께 읽어 Docker가 항상 더 좋은 선택이 아니라 상황에 따른 운영 판단이라는 점을 확인한다.

Visual 2: local 실행과 Docker 실행 비교 표

관점	Local computer	Docker
Compute	host에서 process 실행	image에서 container process 실행
OS/kernel	host OS의 kernel이 직접 process를 관리	container는 host kernel을 공유한다. macOS의 Linux container는 Docker Desktop의 내부 Linux VM 계층을 통해 실행되고, Windows 사용자는 WSL 2 조건을 별도 확인한다.
Storage	host file path 직접 사용	image layer, bind mount, volume
Network	localhost와 host port	container port와 host port binding
Configuration	host env/config file	-e, .env, Compose environment
Observability	terminal log, process status	docker logs, docker ps, exit code
Cleanup	process 종료, 파일 삭제	stop/rm, image/volume 정리

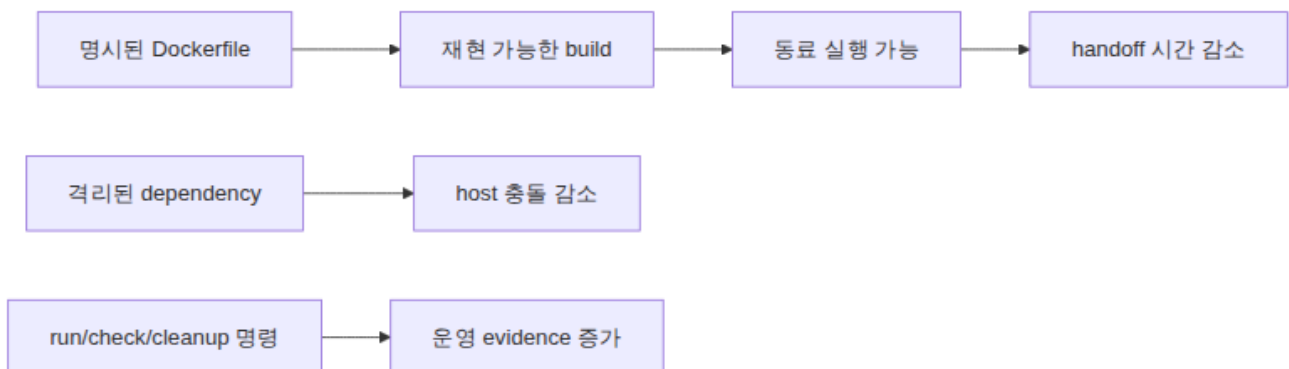
읽는 순서: Docker는 Week 1의 컴퓨팅 구성요소를 없애지 않는다. 각 요소에 더 명확한 경계와 명령을 붙인다.

Docker가 좋아지는 점

Docker의 장점은 "편하다"보다 구체적으로 말해야 한다. 첫째, 실행 조건을 image로 포장해 다른 장비에서 재현하기 쉬워진다. 둘째, runtime과 dependency를 host에 직접 설치하지 않아 충돌을 줄일 수 있다. 셋째, Dockerfile과 Compose를 통해 handoff 문서가 code와 가까워진다.

이 장점은 비용 절감과도 연결된다. cloud resource를 바로 만들기 전에 local container로 실행 조건을 검증하면, 잘못된 설정으로 cloud 환경에서 시간을 낭비하거나 비용을 발생시키는 일을 줄일 수 있다. 단, local 검증이 production 검증을 대체하는 것은 아니다.

Visual 3: Docker benefit map



읽는 순서: Docker의 장점은 명시성에서 시작해 재현성, 협업, 운영 evidence로 이어진다.

Docker가 나빠질 수 있는 점

Docker를 쓰면 책임도 늘어난다. image가 커지면 build/pull 시간이 길어지고 disk를 많이 쓴다. container를 정리하지 않으면 port 충돌과 resource 낭비가 생긴다. secret을 image layer에 넣으면 삭제하기 어렵고 registry로 push될 수 있다. container 내부에서만 발생하는 문제를 분석하려면 docker logs, docker exec, network/volume 확인 같은 추가 관찰 방법이 필요하다.

따라서 Docker는 모든 프로젝트에 자동으로 좋은 선택이 아니다. 특히 아주 작은 일회성 script, 교육 초반의 개념 확인, host hardware를 직접 다루는 작업, GUI 중심 작업은 Docker 도입이 오히려 이해를 어렵게 할 수 있다.

위험 표: Docker 도입으로 생기는 운영 책임

위험	증상	예방/완화
image 비대화	pull/build 시간이 길어짐	.dockerignore, 작은 base image, 불필요 파일 제외
secret 포함	image나 repository에 credential 노출	env var, secret 관리, build context 점검
port 충돌	container는 실행됐지만 접속 실패	docker ps, host port 변경, cleanup
volume 오해	DB 데이터가 사라지거나 남음	named volume과 cleanup 절차 구분
debugging 복잡도	host에서는 보이지 않는 실패	logs, exec, inspect, README troubleshoot

언제 Docker를 쓰지 말아야 하는가

도구 선택은 유행이 아니라 문제 적합성으로 결정한다. Docker는 반복 실행, dependency isolation, handoff, multi-service local environment가 필요한 경우에 강하다. 반대로 실행 조건이 단순하고 한 번만 쓰이며, host와 직접 상호작용해야 하고, container 경계가 학습 목표를 흐리게 만들면 도입을 늦출 수 있다.

Visual 4: Docker 사용 판단표

상황	Docker 사용 권장	Docker 도입 보류 가능
팀원이 같은 앱을 반복 실행	예	
DB/cache 등 여러 service가 필요	예, Compose와 연결	
runtime version 충돌이 잦음	예	
단일 HTML 파일 확인		local server로 충분
일회성 CLI 연습		host command 학습 우선
secret 관리 기준이 없음	기준 수립 후 사용	무작정 image build 금지
host 장치/GUI 직접 의존	신중하게 검토	local 실행이 더 명확할 수 있음

운영 관점 비교 활동

아래 표를 자기 Week 1 앱 또는 표준 실습 앱 기준으로 채운다. 답은 Docker를 무조건 쓰는 방향이어야 할 필요가 없다. 중요한 것은 판단 근거다.

```
## Docker Fit Check
```

```
| 질문 | 내 답 |
| --- | --- |
| 실행 command가 README에 명확한가? | |
| runtime/dependency가 다른 장비에서도 필요할까? | |
| port가 명확한가? | |
| 외부 설정이나 secret이 필요한가? | |
| 데이터가 container 삭제 후 남아야 하는가? | |
| Docker가 handoff 시간을 줄일까? | |
| Docker 도입으로 생기는 새 위험은 무엇인가? | |
```

다음 명령 실습 준비

다음 교시부터는 기본 명령어를 다룬다. 명령어는 외우는 목록이 아니라 상태 확인 도구로 읽는다.

명령	확인하려는 상태
<code>docker version</code>	CLI와 daemon 연결 상태
<code>docker pull</code>	registry에서 image를 받을 수 있는지
<code>docker images</code>	local에 어떤 image가 있는지
<code>docker run</code>	image에서 container를 시작할 수 있는지
<code>docker ps</code>	현재 실행 중인 container가 무엇인지
<code>docker logs</code>	process가 남긴 stdout/stderr evidence
<code>docker stop</code>	실행 중인 container를 멈출 수 있는지
<code>docker rm</code>	종료된 container를 정리했는지

흔한 오해

오해	바로잡기
Docker를 쓰면 production과 완전히 같다.	host OS, network, resource, cloud permission 차이는 여전히 남는다.
Docker container는 OS 전체를 들고 다니는 작은 VM이다.	container는 app/library 중심의 실행 환경이며 host kernel을 공유하는 격리된 process로 이해해야 한다.
Docker는 항상 성능과 비용을 줄인다.	image 크기, build 시간, disk 사용량, 실행 container 수를 관리해야 한다.
container 삭제는 항상 안전하다.	volume과 data lifecycle을 구분하지 않으면 필요한 데이터를 잃을 수 있다.
local 실행은 이제 배울 필요 없다.	Docker 문제도 process, file, network, config evidence로 분석한다.

평가 기준

기준	2점 evidence
비교 관점	compute/storage/network/config/observability 중 3개 이상으로 local과 Docker를 비교했다.
장점 설명	재현성, dependency isolation, handoff 중 2개 이상을 구체적으로 설명했다.
위험 설명	secret, disk, port, volume, debugging 중 2개 이상을 위험으로 기록했다.
도구 선택	Docker 사용/보류 판단을 상황 기준으로 설명했다.
다음 실습 준비	기본 명령어를 상태 확인 목적과 연결했다.

공식/학술 근거 링크

- Docker Docs: Docker overview, <https://docs.docker.com/get-started/docker-overview/> - Docker의 architecture와 container/image 관계를 local 실행과 비교하는 기준이다.
- Docker Docs: Writing a Dockerfile, <https://docs.docker.com/guides/docker-concepts/building-images/writing-a-dockerfile/> - 실행 조건을 Dockerfile instruction으로 명시하는 기준이다.
- OWASP Secrets Management Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html - secret을 image, repository, screenshot에 남기지 않는 보안 판단 기준이다.

다음 연결

다음 교시는 Docker 기본 명령어 첫 실습이다. 각 명령은 "무엇을 확인하는가"라는 질문과 함께 실행하고, 실행 후에는 반드시 중지와 정리까지 기록한다.